

```
In [1]: library(ggplot2)
library(mgcv)
```

Loading required package: nlme

This is mgcv 1.8-31. For overview type 'help("mgcv-package")'.

C3M4 Peer Reviewed Assignment

Outline:

The objectives for this assignment:

1. Observe the difference between GAMs and other regression models on simulated data.
2. Review how to plot and interpret the coefficient linearity for GAM models.

General tips:

1. Read the questions carefully to understand what is being asked.
2. This work will be reviewed by another human, so make sure that you are clear and concise in what your explanations and answers.

```
In [2]: # Load packages
library(ggplot2)
library(mgcv)
```

Problem 1: GAMs with Simulated Data

In this example, we show how to check the validity of a generalized additive model (GAM) (using the `gam()` function) using simulated data. This allows us to try and understand the intricacies of `gam()` without having to worry about the context of the data.

1. (a) Simulate the Data

Let $n = 200$. First, construct three predictor variables. The goal here is to construct a GAM with different types of predictor terms (e.g., factors, continuous variables, some that will enter linearly/parametrically, some that enter transformed).

1. x_1 : A continuous predictor that, we will suppose has a nonlinear relationship with the response.
2. x_2 : A categorical variable with three levels: `s`, `m`, and `t`.
3. x_3 : A categorical variable with two levels: `TRUE` and `FALSE`.

Then, make the response some nonlinear/nonparametric function of $\mathbf{x}$. For our case, use:

$$\log(\mu_i) = \beta_1 + \sin(0.5x_{i,1}^2) - x_{i,2} + x_{i,3}$$

This model is a Poisson GAM. In a realworld situation, we wouldn't know this functional relationship and would estimate it. Other terms are modeled parametrically. The response has normal noise.

Note that:

1. The construction of $\boldsymbol{\mu} = (\mu_1, \dots, \mu_n)^T$ has the linear predictor exponentiated, because of the nature of the link function.
2. We use $\boldsymbol{\mu}$ to construct $\mathbf{y} = (y_1, \dots, y_n)^T$. The assumption for Poisson regression is that the random variable Y_i that generates y_i is Poisson with mean μ_i .
3. `as.integer(as.factor(VARIABLE))` converts the labels of `VARIABLE` to 1, 2, 3,.. so that we can construct the relationship for these factors.

Plot the relationship of $\mathbf{y}$ to each of the predictors. Then, split the data into a training (`train_sim`) and test (`test_sim`) set.

```

In [96]: set.seed(0112358)
n = 200
x = rexp(n)
# create continuous predictor
x1 = sin(x) * (.5*pi)

#convert integers to a factor
x2 = as.factor(sample(c("s", "m", "t"),n,replace=TRUE))
x3 = as.factor(sample(c(TRUE,FALSE),n,replace=TRUE))

# create numeric values for categorical predictor variables
x2_numeric = as.numeric(x2)
x3_numeric = as.numeric(x3)

# input model equations
log_mu = 1 + sin(0.5*x1^2) - x2_numeric + x3_numeric
mu = exp(log_mu)
y = rpois(n,lambda=mu)

par(mfrow=c(1,1))
plot(x1,y,main="Y as a function of x1(Nonlinear continuous predictor)")
plot(x2,y,main="Y as a function of x2(Cateogrical variable)")
plot(x3,y,main="Y as a function of x3(cateogrical variable)")

# create data frame for splitting value and conducting regression analysis
df = data.frame(y,x1,x2,x3)

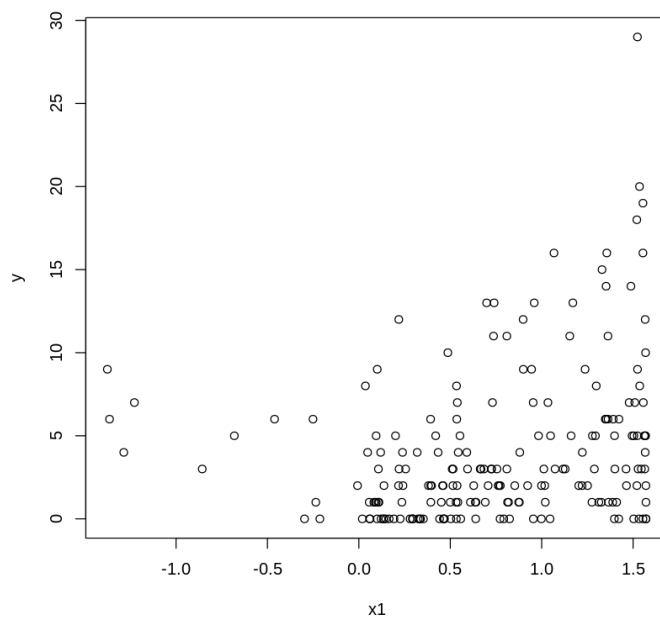
# split into trainign and testing datasets
n2 = floor(0.8*nrow(df))
index = floor(sample(seq_len(nrow(df)),size=n2))

train_sim = df[index, ]
test_sim = df[-index, ]

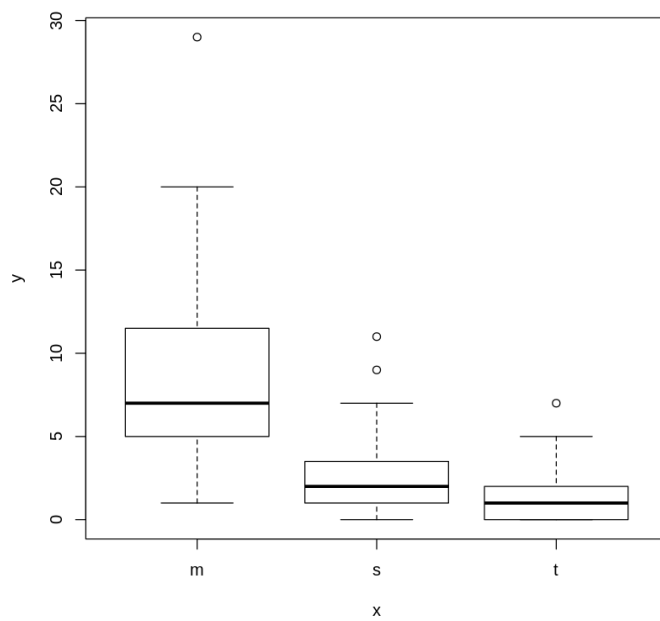
```

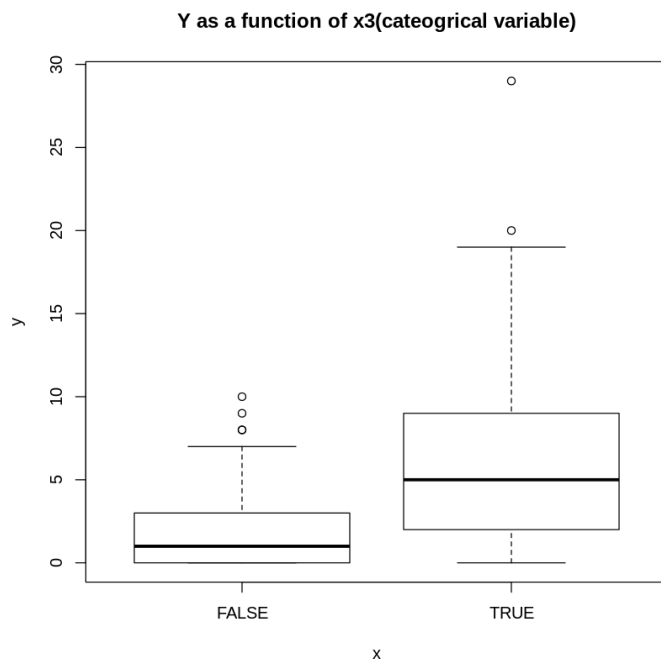
Min. 1st Qu. Median Mean 3rd Qu. Max.
0.01203 0.31197 0.66638 1.10123 1.65549 5.98529

Y as a function of x1(Nonlinear continuous predictor)



Y as a function of x2(Categorical variable)





1. (b) Other Regression Models

Before jumping straight into GAMs, let's test if other regression models work. What about a regular linear regression model with ordinary least squares, and a generalized linear model for Poisson regression?

First fit a linear regression model to your `train_sim` data. We know that all of the predictors were used to make the response, but are they all significant in the linear regression model? Explain why this may be.

Then fit a Generalize Linear Model (GLM) to the `train_sim` data. Plot three diagnostic plots for your GLM:

1. Residual vs. log(Fitted Values)
2. QQPlot of the Residuals
3. Actual Values vs. Fitted Values

Using these plots, determine whether this model is a good fit for the data. Make sure to explain your conclusions and reasoning.

```
In [102]: # Fit a LM model to the data
mod_1 = lm(y~x1+x2+x3,data=train_sim)
summary(mod_1)

# Fit a GLM model to the data
glm = glm(y~x1+x2+x3,data=train_sim,family = poisson)
summary(glm)
pchisq(206.87,155,lower=FALSE) # pull data from the model output

#residual plot
r = resid(glm)
log_f = log(fitted(glm))
plot(log_f,r,main="Residual v. Log(Fitted)")
abline(h=0,col="green",lwd=2)
## qqplot
qqnorm(r,main="QQ Plot")
abline(0,1,col="red",lwd=2)

#fitted vs actual
plot(fitted(glm),glm$y,main="Fitted v. Actual")
abline(0,1,col="orange",lwd=2)
```

Call:

```
lm(formula = y ~ x1 + x2 + x3, data = train_sim)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-5.884	-1.633	-0.118	1.287	17.486

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.2039	0.5770	9.019	6.83e-16 ***
x1	1.7378	0.3630	4.787	3.92e-06 ***
x2s	-5.1977	0.5542	-9.379	< 2e-16 ***
x2t	-7.1601	0.5766	-12.417	< 2e-16 ***
x3TRUE	3.6624	0.4643	7.888	5.15e-13 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.883 on 155 degrees of freedom

Multiple R-squared: 0.6549, Adjusted R-squared: 0.646

F-statistic: 73.54 on 4 and 155 DF, p-value: < 2.2e-16

Call:

```
glm(formula = y ~ x1 + x2 + x3, family = poisson, data = train_sim)
```

Deviance Residuals:

	Min	1Q	Median	3Q	Max
	-2.5275	-1.0260	-0.1505	0.5061	3.9517

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.12755	0.10838	10.404	< 2e-16 ***
x1	0.43220	0.06733	6.419	1.37e-10 ***
x2s	-0.94929	0.09503	-9.989	< 2e-16 ***
x2t	-1.82961	0.13684	-13.371	< 2e-16 ***
x3TRUE	0.93207	0.08936	10.430	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

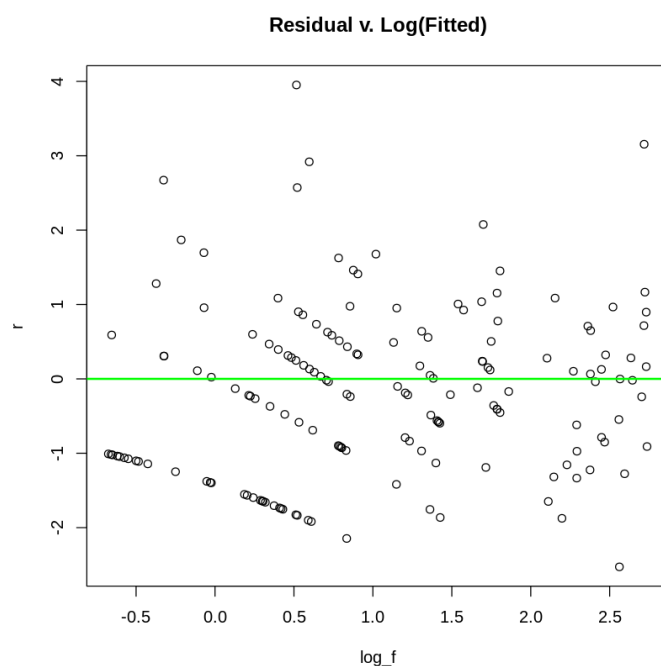
Null deviance: 757.44 on 159 degrees of freedom

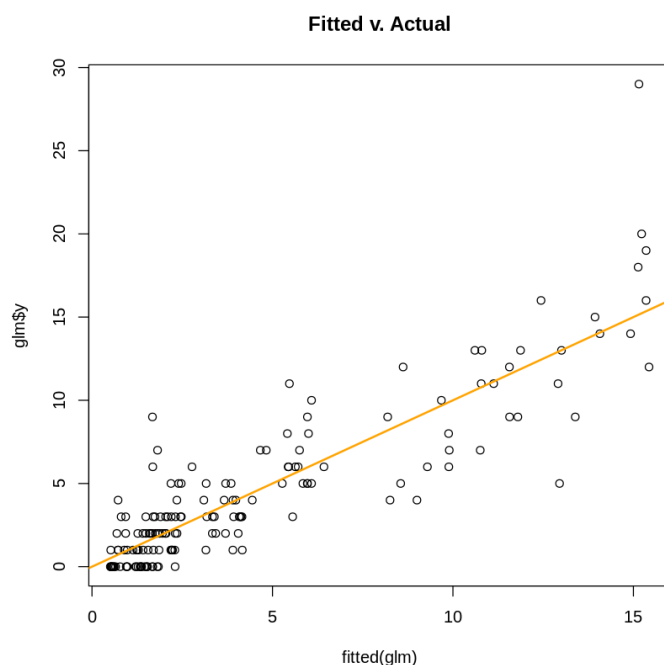
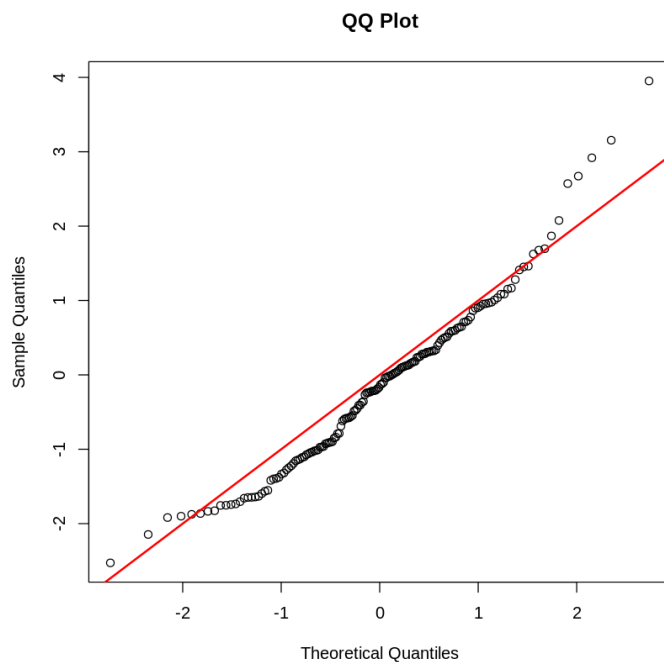
Residual deviance: 206.87 on 155 degrees of freedom

AIC: 638.86

Number of Fisher Scoring iterations: 5

0.00339948471808891





All of the predictors in the linear model are statistically significant. We expect this to occur in a model with few predictors as each predictor provides unique explanatory power, regardless of goodness of fit.

The p-value associated with the GLM residual deviance is .003, thus we reject the hypotheses that our model fits well enough. This is supported by our diagnostic plots. The residuals v. fitted values plot shows residuals are unevenly distributed about 0, with values converging below zero as $\log_f$ increases. The clustering or correlation structure suggests the presence of heteroscedasticity.

In the QQ plot we observe pronounced variations from normality, especially in the lower tailed region.

The fitted v. actual plot shows strong evidence of underfitting.

1. (c) Looking for those GAMs

Now, it's time to see how a generalized additive model (GAM) performs! Fit a GAM to the data. Construct the same three plots for your GAM model. Do these plots look better than those of the GLM?

```
In [105]: # Fit a GAM model to the data
gam = gam(y~s(x1)+ x2 + x3,data=train_sim,family=poisson)
#plot(gam)
summary(gam)
r = resid(gam)
fitted = fitted(gam)
# Construct the three specified plots

#residual vs fitted
plot(fitted,r,main="residual v. fitted plot")
abline(h=0,col="green",lwd=2)
## qqplot
qqnorm(r,main="qqplot")
abline(0,1,col="red",lwd=2)

#fitted vs actual
plot(fitted,gam$y)
abline(0,1,col="orange",lwd=2)
```


Family: poisson
Link function: log

Formula:
 $y \sim s(x1) + x2 + x3$

Parametric coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.43734	0.08604	16.71	<2e-16 ***
x2s	-0.97813	0.09609	-10.18	<2e-16 ***
x2t	-1.86087	0.13697	-13.59	<2e-16 ***
x3TRUE	0.91989	0.08910	10.32	<2e-16 ***

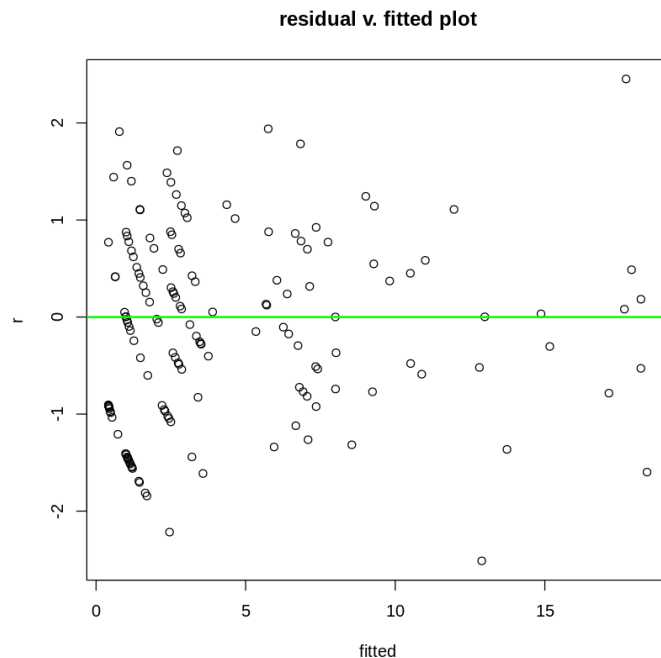
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

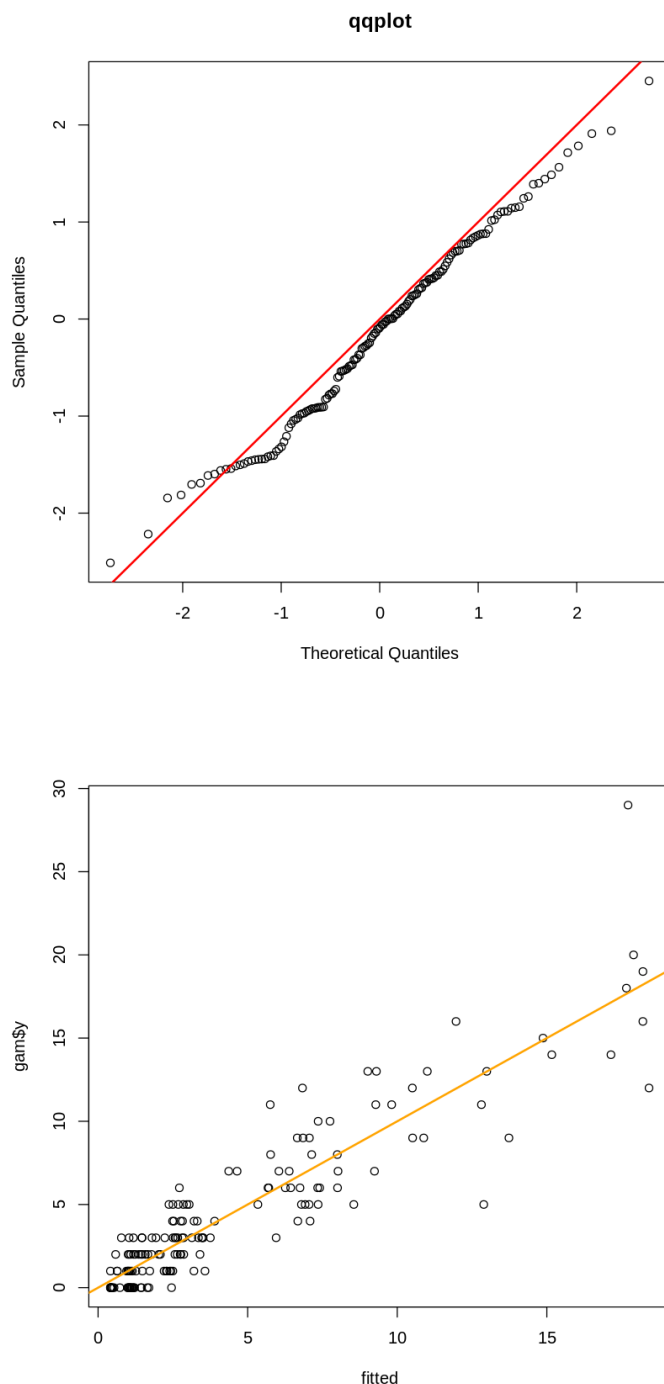
Approximate significance of smooth terms:

	edf	Ref.df	Chi.sq	p-value
s(x1)	3.223	3.962	91.47	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.818 Deviance explained = 79.6%
UBRE = 0.053975 Scale est. = 1 n = 160





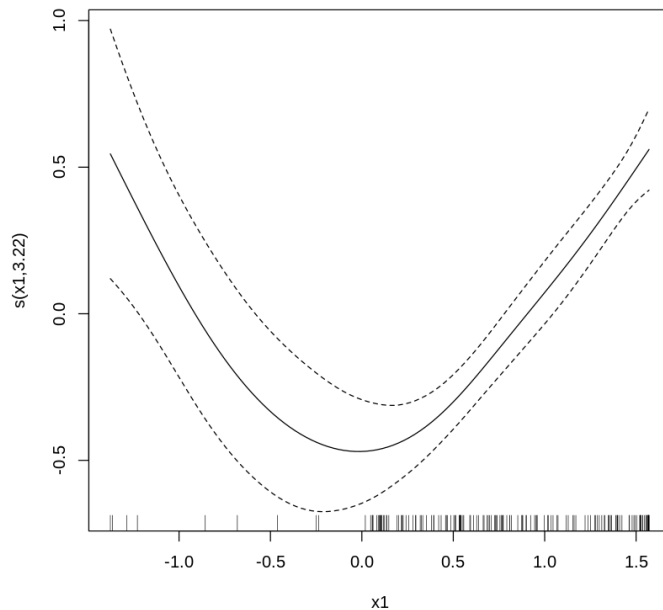
The diagnostic plots look marginally better which is expected based on the improvements in r-squared and residual deviance. The primary differences can be observed in the qq and fitted v. actual plots. In the qqplot there is less deviation from normality in the lower tail region and in the fitted v. actual plot we see less underfitting as the predicted values, on average, are closer to the true values.

1. (d) Interpreting GAMs

We made a GAM model! However GAMs are harder to interpret than regular linear regression models. How do we determine if a GAM model was necessary? Or, in other words, how do we determine if our predictors have a linear relationship with the response?

Use the `plot.gam()` function in the `mgcv` library to plot the relationship between y and x_1 . Recall that x_1 entered our model as $\sin(0.5x_{i,1}^2)$, and we plotted that relationship in **1.(a)**. Does your plot confirm this relationship?

In [106]: `plot.gam(gam)`



The plot confirms the sinusoidal relationship that we entered in 1.(a)

1.(e) Model comparison

Compute the mean squared prediction error (MSPE) for each of the three models above (regression model, GLM, and GAM). State which model performs best according to this metric.

Remember, the MSPE is given by

$$MSPE = \frac{1}{k} \sum_{i=1}^k (y_i^* - \hat{y}_i^*)^2$$

where y_i^* are the observed response values in the test set and $\hat{y}_i^*$ are the predicted values for the test set (using the model fit on the training set).

```
In [108]: #mspe for lm
linear = mod_1
pred_linear = predict(linear, newdata=test_sim)
MSPE_linear = mean((test_sim$y - pred_linear)^2)

#mspe for glm
pred_glm = predict(glm, newdata=test_sim, type="response")
MSPE_glm = mean((test_sim$y - pred_glm)^2)

# mspe for gam
pred_gam = predict(gam, newdata=test_sim, type="response")
MSPE_gam = mean((test_sim$y - pred_gam)^2)

cat("MSPE linear model:", {MSPE_linear}, "\n",
    "MSPE GLM model:", {MSPE_glm}, "\n",
    "MSPE GAM model:", {MSPE_gam})
```

```
MSPE linear model: 5.809813
MSPE GLM model: 2.360267
MSPE GAM model: 2.048653
```

GAM model performs best with MSPE of 2.05. The superior performance of the GAM model occurs because it models the non-parametric and parametric relationships leading to less error and more precise predictions.

Problem 2 Additive models with the advertising data

The following dataset contains measurements related to the impact of three advertising medias on sales of a product, P . The variables are:

- youtube : the advertising budget allocated to YouTube. Measured in thousands of dollars;
- facebook : the advertising budget allocated to Facebook. Measured in thousands of dollars; and
- newspaper : the advertising budget allocated to a local newspaper. Measured in thousands of dollars.
- sales : the value in the i^{th} row of the sales column is a measurement of the sales (in thousands of units) for product P for company i .

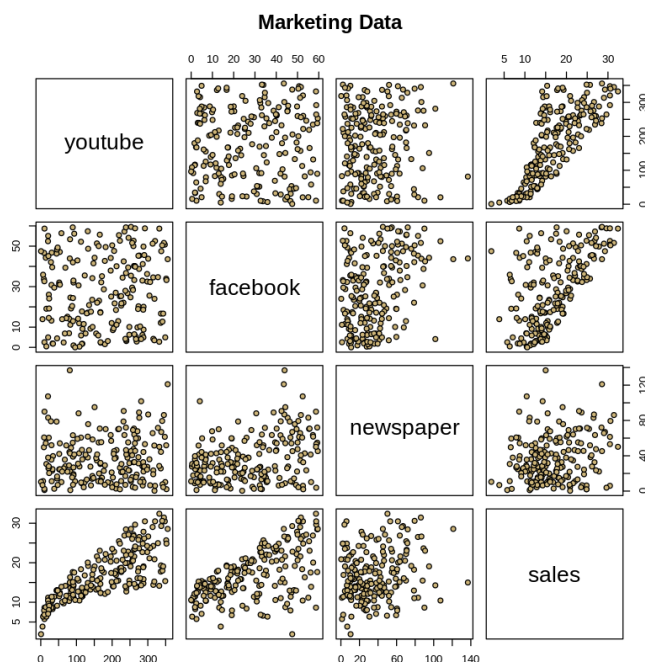
The advertising data treat "a company selling product P " as the statistical unit, and "all companies selling product P " as the population. We assume that the $n = 200$ companies in the dataset were chosen at random from the population (a strong assumption!).

First, we load the data, plot it, and split it into a training set (`train_marketing`) and a test set (`test_marketing`).

```
In [22]: # Load in the data
marketing = read.csv("marketing.txt", sep = "")
head(marketing)
pairs(marketing, main = "Marketing Data", pch = 21,
      bg = c("#CFB87C"))
```

A data.frame: 6 × 4

	youtube	facebook	newspaper	sales
	<dbl>	<dbl>	<dbl>	<dbl>
1	276.12	45.36	83.04	26.52
2	53.40	47.16	54.12	12.48
3	20.64	55.08	83.16	11.16
4	181.80	49.56	70.20	22.20
5	216.96	12.96	70.08	15.48
6	10.44	58.68	90.00	8.64



```
In [23]: set.seed(177) #set the random number generator seed.
n = floor(0.8 * nrow(marketing)) #find the number corresponding to 80% of the data
index = sample(seq_len(nrow(marketing)), size = n) #randomly sample indices to be included i
n the training set

train_marketing = marketing[index, ] #set the training set to be the randomly sampled rows of
the dataframe
test_marketing = marketing[-index, ] #set the testing set to be the remaining rows
dim(test_marketing) #check the dimensions
dim(train_marketing) #check the dimensions
```

40 · 4

160 · 4

2.(a) Let's try a GAM on the marketing data!

Note that the relationship between `sales` and `youtube` is nonlinear. This was a problem for us back in the first course in this specialization, when we modeled the data as if it were linear. In the last module, we focused on modeling the relationship between `sales` and `youtube`, omitting the other variables. Now it's time to include the additional predictors.

Using the `train_marketing` fit an additive model to the data and store it in `gam_marketing`. Produce the relevant added variable plots using `plot(gam_marketing)`. Comment on the fit of the model.

```
In [109]: gam_no_smoothing = gam(sales ~ youtube + newspaper + facebook, data=train_marketing)
sum_no_smoothing = summary(gam_no_smoothing)
cat("Explained deviance of GAM model without smoothing:",{round(sum_no_smoothing$dev.expl,
3)},"\n")
gam_marketing = gam(sales ~ s(youtube) + newspaper + facebook, data=train_marketing)
sum_marketing = summary(gam_marketing)
cat("Explained deviance of GAM model with smoothing:",{round(sum_marketing$dev.expl,3)},"\n")
change = ((sum_marketing$dev.expl - sum_no_smoothing$dev.expl) / sum_no_smoothing$dev.expl)*100
cat("Explained deviance of GAM model with smoothing is", {round(change,3)},"higher than GAM model without smoothing")
summary(gam_marketing)
plot(gam_marketing)
```

Explained deviance of GAM model without smoothing: 0.897

Explained deviance of GAM model with smoothing: 0.932

Explained deviance of GAM model with smoothing is 3.87 higher than GAM model without smoothing

Family: gaussian

Link function: identity

Formula:

sales ~ s(youtube) + newspaper + facebook

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	11.264013	0.279555	40.293	<2e-16 ***
newspaper	0.004403	0.005811	0.758	0.45
facebook	0.186582	0.008530	21.874	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

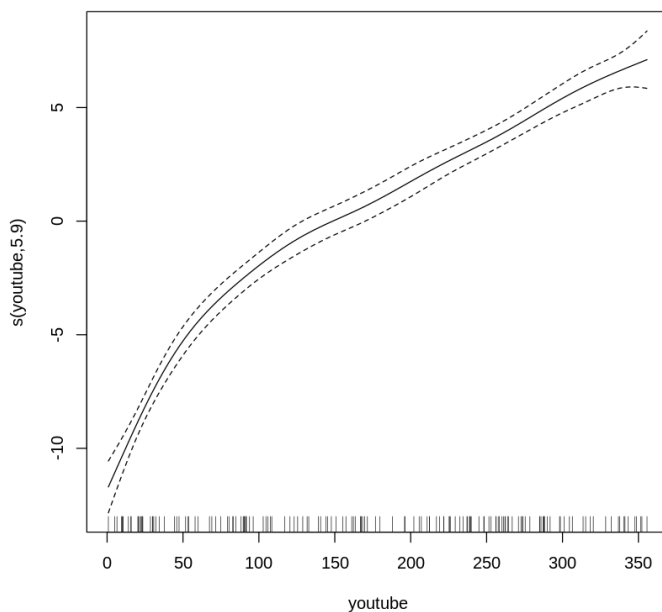
Approximate significance of smooth terms:

	edf	Ref.df	F	p-value
s(youtube)	5.9	7.047	202.1	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.929 Deviance explained = 93.2%

GCV = 2.9718 Scale est. = 2.8065 n = 160



The GAM model with smoothing has a higher explained deviance, increasing 3.87% over the same model without smoothing. The plot shows that the relationship between sales and youtube is non-linear. In the interval Youtube{0,75} there is a pronounced increase in sales for each unit increase in youtube advertsing. In the interval Youtube{75,350} the trend is less pronounced as sales increase at a decreasing rate.

2.(b) Semiparametric modeling of the marketing data

Refit the additive model based on your results from 2.(a). That is, if any predictors above should enter linearly, refit the model to reflect that. If any predictors are statistically insignificant, remove them from the model. Store your final model in `semiparametric_marketing`.

```
In [58]: semiparametric_marketing = gam(sales ~ s(youtube) + facebook, data=train_marketing)
```

I removed the newspaper variable since it was statistically insignificant.

```
In [ ]:
```

2.(c) Model comparisons

Now, let's do some model comparisons on the test data. Compute the mean squared prediction error (MSPE) on the `test_marketing` data for the following three models:

- `gam_marketing` from 2.(a)
- `semiparametric_marketing` from 2.(b)
- `lm_marketing`, a linear regression model with `sales` is the response and `youtube` and `facebook` are predictors (fit on the `train_marketing` data).

State which model performs based according to this metric.

```
In [110]: #mspe for gam
pred_gam = predict(gam_marketing, newdata=test_marketing)
MSPE_gam = mean((test_marketing$sales - pred_gam)^2)

#mspe for semiparametric
pred_semi = predict(semiparametric_marketing, newdata=test_marketing)
MSPE_semi = mean((test_marketing$sales - pred_semi)^2)

# mspe for lm
lm_marketing = lm(sales ~ youtube + facebook, data=train_marketing)
pred_lm = predict(lm_marketing, newdata=test_marketing)
MSPE_lm = mean((test_marketing$sales - pred_lm)^2)

c(MSPE_gam,MSPE_semi,MSPE_lm)

((MSPE_semi - MSPE_lm)/MSPE_lm)*100
((MSPE_semi - MSPE_gam)/MSPE_gam)*100
```

```
3.40994916862721 · 3.29744826352071 · 4.19770116857387
-21.4463314299957
-3.29919595698238
```

The semiparametric model has the lowest MSPE. On average, it outperforms the linear model by 21% and the GAM model by 3%, in terms of prediction accuracy.

```
In [ ]:
```